

Detaillierter Projektplan – Individualphase

Projekt: Echtzeit-Tsunamis – Interaktive Visualisierung mit OpenGL

Team: Mika Brückner, Yannik Köllmann, Jan Vogt

Tsunami Lab · Sommersemester 2026

1 Überblick und Motivation

Unser bestehender Solver löst die zweidimensionalen Flachwassergleichungen (WavePropagation2d, ungesplittete X-/Y-Sweeps auf Basis des F-Wave-Verfahrens) bereits zuverlässig und mit Bathymetrikopplung. Bislang ist der Ablauf jedoch ein reiner Batch-Prozess: Szenario vorkonfigurieren, rechnen lassen, Ergebnisse als CSV/NetCDF schreiben und anschließend in ParaView ansehen. Zwischen Eingabe und Ergebnis liegt spürbare Wartezeit, und am Ende steht ein fixes, nicht interaktives Ereignis.

Ziel der Individualphase ist es, denselben Solver in eine interaktive Echtzeit-Umgebung zu überführen. Der Nutzer wählt ein geografisches Gebiet, die Bathymetrie wird automatisch geladen, Meeresboden und Wasseroberfläche werden dreidimensional gerendert, und ein Tsunami lässt sich per Klick physikalisch plausibel auslösen. Statt einer naiven Wassersäule erzeugen wir die initiale Auslenkung über ein vereinfachtes Okada-Modell (Bodenverwerfung), das konsistent zur bereits vorhandenen deltaXPsi-Bathymetrikopplung des Solvers ist.

Die technische Ausgangslage ist günstig: Die Methoden `getHeight()` und `getBathymetry()` liefern rohe `t_real*`-Zeiger, die sich direkt in OpenGL-Vertex-Buffer-Objects hochladen lassen. Zwei der drei Teammitglieder bringen bereits OpenGL-Erfahrung aus dem Studium mit. Das Team hat sich verbindlich auf diese Richtung festgelegt.

2 Ziele

Hauptziel. Überführung des bestehenden Batch-SWE-Solvers in eine interaktive, echtzeitfähige OpenGL-Anwendung, in der Tsunami-Ereignisse live ausgelöst und beobachtet werden können – ohne Konfigurationsdatei und ohne Zwischenspeicherung.

Teilziele.

- **Echtzeit-Rendering.** Dreidimensionale Darstellung von Bathymetrie und Wasseroberfläche, kontinuierlich aktualisiert aus dem laufenden Solver.
- **Physikalisch sinnvolle Interaktion.** Klickbasierte Auslösung über ein vereinfachtes Okada-Modell statt einer beliebigen Wassersäule; einstellbare Parameter (Erdbebenstärke, Herdtiefe, Ausrichtung der Verwerfung).
- **Datenanbindung.** Automatisches Laden realer Bathymetrie über die GEBCO-Schnittstelle anhand eines gewählten Gebiets.
- **Realbezug.** Ladbare, parametrisierte Szenarien echter Ereignisse (Tohoku 2011, Chile 1960).
- **Entkopplung von Rechnung und Darstellung.** Der Solver rechnet in einem Hintergrund-Thread; Ergebnisse werden über einen Doppelbuffer in die Visualisierung gestreamt, ohne das Rendering zu blockieren.
- **Codequalität.** Erhalt der bestehenden Standards: Catch2-Unittests, Valgrind, SCons-Build, Sphinx-/Doxygen-Dokumentation und GitHub-Actions-CI.

Nicht-Ziele (Abgrenzung). Keine geophysikalisch vollständige Okada-Implementierung mit allen neun Quellparametern, wir starten mit einer Gauß-förmigen Bodenverschiebung und nähern uns dem Okada-Modell schrittweise. Keine persistente Speicherung der Ergebnisse, da alles live dargestellt wird. Keine GPU-Parallelisierung des Solvers.

3 Arbeitspakete

WP	Titel	Verantwortlich	Inhalt
WP1	Architektur & Build-Integration	Mika	Einbindung von OpenGL (GLFW/GLAD bzw. äquivalent) in den bestehenden SCons-/C++11-Build; Threading-Modell zur Trennung von Solver- und Render-Schleife; Schnittstellendefinition zwischen Solver und Renderer.
WP2	Render-Pipeline	Yannik	Aufbau des 3D-Meshes aus getHeight()/getBathymetry(); VBO-Upload der t_real*-Daten; Kamera- und Steuerungslogik; Beleuchtung; geteilte Colormap mit eigener Skalierung ober- und unterhalb von Null.
WP3	Bathymetrie-Anbindung (GEBCO)	Jan	Gebietsauswahl in der Oberfläche; Abruf und Parsen der GEBCO-NetCDF-Daten; Resampling auf das Solver-Gitter; lokal gebündelter Fallback-Datensatz.
WP4	Interaktive Auslösung	Mika	Picking (Bildschirm- → Gitterkoordinate); Gauß-Verschiebung auf Zellen mit negativer Bathymetrie als vereinfachtes Okada-Modell; Parameter-Eingabe (Stärke, Herdtiefe, Ausrichtung); Kopplung an die Wasserhöhe analog deltaXPsi.
WP5	Szenario-System	Jan	Vordefinierte Ereignisse (Tohoku 2011, Chile 1960) auf Basis realer Erdbebenparameter; ladbar als Szenario, live ablaufend.
WP6	Solver-Renderer-Kopplung	Yannik / Mika	Hintergrund-Thread für die Zeitintegration; Doppelbufferung der Höhenfelder; kontinuierliches Streaming der Ergebnisse in die Visualisierung; Sub-Stepping, falls der Solver dem Rendering vorseilt.
WP7	Dokumentation, Test & Bericht	alle	Catch2-Tests für neue Komponenten; RST-Bericht; Sphinx-/Doxygen-Doku via GitHub Pages; Vorbereitung der Abschlusspräsentation.

Die personelle Zuordnung ist als Schwerpunkt zu verstehen, die rendering-lastigen Pakete liegen bei den beiden Teammitgliedern mit OpenGL-Vorerfahrung.

4 Meilensteine

MS	Termin	Kriterium
M1	Mitte Woche 1	OpenGL-Fenster rendert die statische Bathymetrie aus den vorhandenen Solver-Daten (VBO-Upload, Kamera, Colormap).
M2	Ende Woche 1	Wasseroberfläche animiert sich live aus dem im Hintergrund laufenden Solver (Threading + Doppelbuffer funktionieren).
M3	Ende Woche 2	Klick auf das Gebiet löst eine Bodenverschiebung aus; eine Welle läuft stabil los; GEBCO-Gebiet ladbar; Okada-Parameter (Stärke, Tiefe, Ausrichtung) wirken.
M4	Ende Woche 3	Szenario(en) laufen live; Anwendung stabil; Bericht, Tests und Doku abgeschlossen; Verteidigung vorbereitet.

5 Zeitplan

Der Zeitplan erstreckt sich über drei Wochen ab dem 18.06.2026.

Woche	Zeitraum	Schwerpunkt	Pakete
1	18.-25.06.	Setup, Architektur, statisches und live Rendering (Threading & Doppelbuffer)	WP1, WP2, WP6
2	25.06.-02.07.	Interaktion (Picking, Gauß-/Okada-Verschiebung), GEBCO-Anbindung	WP4, WP3
3	02.-08.07.	Szenario(en), Stabilisierung, Tests, Bericht, Verteidigung	WP5, WP7

6 Risiken und Gegenmaßnahmen

Risiko	Auswirkung	Gegenmaßnahme
Knappes Drei-Wochen-Zeitfenster	Nicht alle Features umsetzbar	Strikte Priorisierung: Kern (3D-Rendering, Live-Solver, interaktiver Klick) zuerst; GEBCO-Live-Abruf und zusätzliche Szenarien als Ausbaustufe, falls Zeit bleibt.
GEBCO-Anbindung aufwendiger als gedacht (API, NetCDF, Netzabhängigkeit)	Verzögerung in WP3	Lokal gebündelte Bathymetrie-Datensätze als Fallback; Gebietsauswahl zunächst auf vorbereitete Ausschnitte beschränken.
Echtzeit-Performance: Solver hält mit dem Rendering nicht Schritt	Ruckelnde oder verzögerte Darstellung	Entkopplung über Hintergrund-Thread und Doppelbuffer; Sub-Stepping; Reduktion der Gitterauflösung im

Risiko	Auswirkung	Gegenmaßnahme
		interaktiven Modus.
Okada-Modell numerisch/geophysikalisch komplex	Verzögerung in WP4	Inkrementeller Ansatz: zuerst Gauß-Verschiebung auf Zellen mit negativer Bathymetrie, dann schrittweise Annäherung an Okada.
OpenGL-Einarbeitung des dritten Teammitglieds	Engpass bei rendering- lastigen Paketen	Pair-Programming; klare Schnittstellen; rendering- unabhängige Pakete (WP3, WP4-Physik) parallel bearbeitbar.

7 Erwartete Ergebnisse

- Lauffähige interaktive Anwendung, die reale Bathymetrie lädt, dreidimensional rendert und Tsunami-Ereignisse per Klick physikalisch plausibel auslöst.
- Mindestens ein ladbares Realszenario (Tohoku 2011 bzw. Chile 1960).
- In den bestehenden SCons-/C++11-Build integrierte, getestete und dokumentierte Erweiterung des Solvers ohne Verlust der bisherigen Codequalität.
- Detaillierter Abschlussbericht (RST/Sphinx) und 15-minütige Verteidigung.

Der Mehrwert gegenüber dem bisherigen Batch-Ablauf: kein Video, keine Aufzeichnung, keine Konfigurationsdatei. Ein Klick – der Ozean reagiert.